

Chat Application — Microservices Messaging Platform

AGISIT Project

Management and Administration of IT Infrastructures and Services

Group nr.: 60

ist1119213: Antero Cabral Marques Morgado

ist1119203: António Miguel Duarte Palma

MSc Computer Science and Engineering
IST-ALAMEDA

2025/2026

Chapter 1

Introduction

The primary goal of this project was to design, implement, and operate a realistic tiered, microservices-based web application within a cloud environment. The specific application developed is a functional instant messaging web-app, enabling users to register, log in, and exchange messages directly and within group chats. This solution addresses core challenges of modern distributed systems, including service decomposition, orchestration, and continuous operation.

The project was structured into two main stages. Stage I established the **Base Solution**, focusing on developing the core instant messaging functionality and deploying the multi-tiered architecture (Frontend, Backend, and Data Store) onto a self-hosted **Kubernetes** cluster. This deployment was entirely automated using Infrastructure as Code (IaC) tools. Furthermore, essential observability was configured using Prometheus and Grafana to provide real-time performance monitoring.

Stage II focused on providing advanced functionality through the implementation of a Continuous Integration/Continuous Deployment (CI/CD) pipeline. We leveraged GitLab package new Docker images, and deploy the updated microservices to the Kubernetes cluster upon every commit to the code repository. This enhancement ensures rapid, reliable, and automated delivery of new features.

Project Video Link The video demonstration of the deployment process and the functional application is available at: <https://www.youtube.com/watch?v=uqSPkXvHev0>

Chapter 2

System Design and Architecture

The base system is designed using a Microservices architecture, structured around four main logical components: the Frontend, Backend Services, the Data Store, and the Monitoring Server.

2.1 Frontend

The client-side implementation handles user interaction and exposes the web interface. It acts as the entry point (ingress) and communicates with the Backend Services via an HTTP requests. Coded with HTML, Javascript and CSS.

2.2 Backend Services

Divided in auth-users, groups(chats), messages and contacts, these services provide the core business logic for the instant messaging application. This layer is designed to be stateless and horizontally scalable. To provide real time messages, WebSockets were used with Socket.io. Implemented using Express.js running on Node.js.

2.3 Database

Each service has its own PostgreSQL database, which it is responsible for reading and writing its own. So there is a database for users, messages and read messages, groups(chats) and contacts.

2.4 Monitoring Server

This component provides necessary insights into the system's health. It collects metrics exposed by the microservices (via Prometheus) and displays them in real-time dashboards (using Grafana) to monitor application behavior and performance. It tracks metrics such as CPU Usage by Service, Memory Usage by Service (MB), HTTP Request Rate and Duration (Latency), Service Health, Online Users,

Total Messages Sent, Messages Sent Rate (per minute), Messages Sent by Group, Total Messages Received/Read, Total User Registrations and Logins.

2.5 Infrastructure

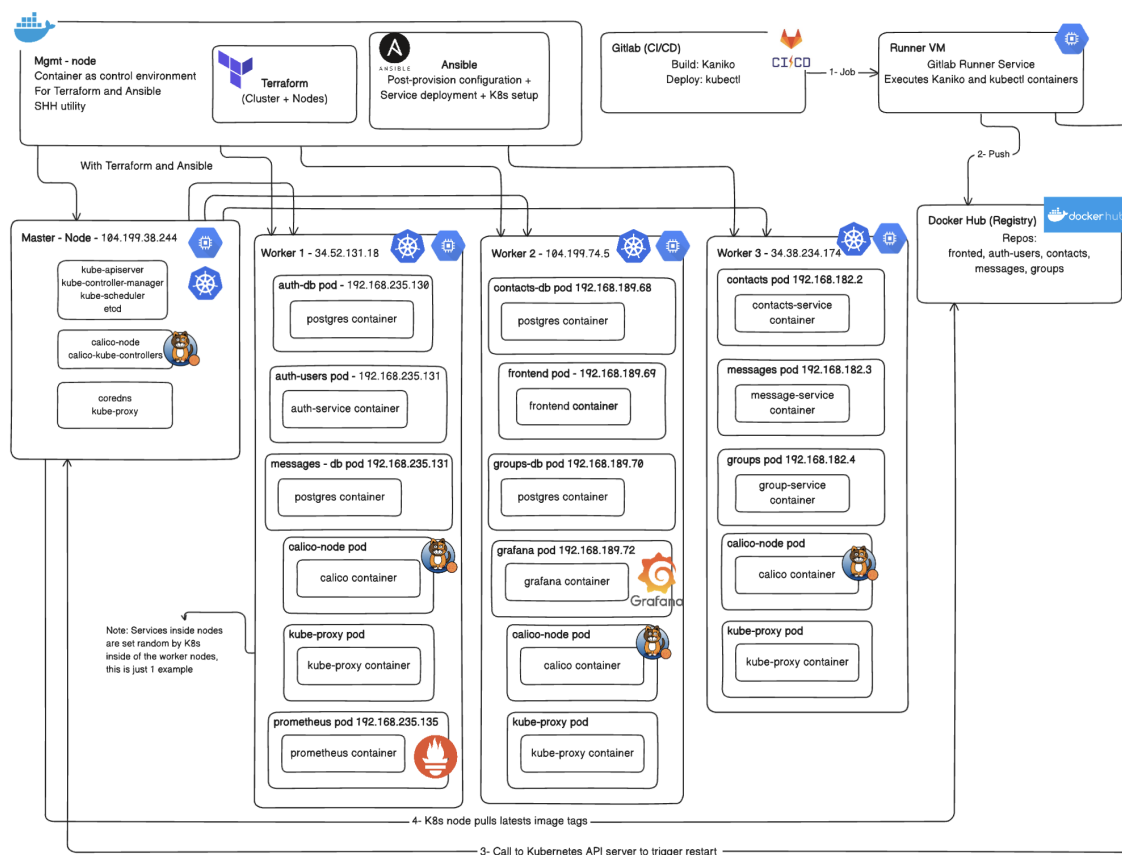
The system is deployed on a self-hosted Kubernetes cluster provisioned with IaC tools (Ansible/Terraform playbooks are available in the repository). The application is organized as Kubernetes Deployments and Services: one frontend and four backend microservices (auth-users, contacts, messages, groups). Each backend is paired with a PostgreSQL database Deployment exposed by a Service.

Prometheus and Grafana run in-cluster for monitoring (Prometheus PVC requests 5, Gi). Prometheus scrapes each service at /metrics and collects application and system metrics (CPU, memory, request rate, latency, health). Sensitive values (DB passwords and JWT secret) are stored in the app-secrets Kubernetes Secret.

Services talk to each other by cluster DNS (for example, `http://auth-users:8081`).

The cluster is composed by 1 master node + 2–3 worker nodes (each 2 vCPU, 4–8 GB RAM).

2.6 Diagram of the whole system



Chapter 3

CI/CD Pipeline (Advanced Component)

The Continuous Integration and Continuous Deployment (CI/CD) pipeline is implemented using **GitLab CI** with a self-hosted **GitLab Runner** deployed on **Google Cloud Platform (GCP)**. This setup automates the complete build and deployment workflow for all project components, ensuring consistency and reliability throughout the process.

The GitLab Runner operates on a dedicated GCP virtual machine configured with Docker and registered to the GitLab instance. It executes pipeline jobs within isolated containers, providing a reproducible and portable execution environment.

To maintain security, environment variables are defined within GitLab to store sensitive credentials and configuration data. These include Docker Hub authentication details for pushing images and the Kubernetes configuration used during the deployment stage.

During the *build stage*, **Kaniko** is used to build Docker images for each service without requiring Docker-in-Docker privileges. The resulting images are pushed to Docker Hub and tagged both as `latest` and with the corresponding commit hash to ensure traceability between source code and deployed versions.

The *deploy stage* uses **kubectl** to connect to the Kubernetes master node and update deployments with the new image tags. Once the update is applied, Kubernetes automatically pulls the new images and restarts the pods across the worker nodes, ensuring the latest versions are active in the cluster.

This configuration ensures that every commit to the `master` branch triggers a fully automated pipeline, continuously building, publishing, and deploying the latest application state to the Kubernetes environment.

Chapter 4

Latency Evaluation

4.1 Evaluation

This section evaluates the system's throughput and latency using ApacheBench (ab). Each test sent 1000 requests (or 500 for internal services) with 10 concurrent clients.

Throughput represents the number of successful requests per second.

- Frontend: 116.8 req/s, no failures – best performance, serving static content.
- Register: 112.4 req/s, all non-2xx – stable but requests invalid (e.g., duplicate data).
- Messages: 44.4 req/s, 920 failed – lower scalability, likely due to DB or token validation overhead.
- Contacts and Groups: ~ 9 req/s – highest processing time (≈ 1 s per request).

Throughput decreases as request complexity increases, reflecting database access and authentication costs.

Latency

Average latencies followed the same trend:

- Frontend: 85 ms mean, 95th p 84 ms.
- Register: 89 ms mean, stable responses.
- Messages: 225 ms mean, up to 2.6 s in outliers.
- Contacts/Groups: ≈ 1.0 – 1.1 s mean, with some exceeding 3 s.

Simple endpoints maintain low latency; database-dependent ones vary more under load.

Stateless services scale efficiently, while stateful ones show clear performance limits. Future improvements could include query optimization, caching, and asynchronous processing.

Although only tested at a fixed concurrency level (10 clients), results indicate that increased concurrency would mainly affect database-intensive endpoints.